

GAME-OPS SYSTEM

Multiplayer Action Game | Weekend Event Operations | Rule-Based Python + FastAPI + Web Dashboard

1. Problem Understanding

During a live weekend event, thousands of players submit match results simultaneously. The ops team needs a system that: (a) builds a fair leaderboard while isolating fraudulent scores, (b) detects cheaters without requiring ML training data, (c) groups players into low-latency, skill-balanced lobbies, and (d) explains how the system scales from 2,000 to 200,000+ concurrent players.

2. Assumptions

Area	Assumption
Data source	CSV / REST POST — no real-time stream at prototype level
Thresholds	Manually tuned to sample data; configurable per game meta
Fair leaderboard	Flagged players shown as "under_review" — do NOT displace clean ranks
Matchmaking	Static bucketing sufficient at prototype; live queue is a prod enhancement
Persistence	In-memory Pandas DataFrame; prod uses PostgreSQL + Redis
Auth / rate-limit	Not implemented at prototype; required in production

3. Data Structure & Database Design

Two logical tables (implemented as Pandas DataFrames; mapped to PostgreSQL in production):

matches (raw telemetry)

Field	Type	Note
player_id / match_id	VARCHAR	Composite primary key — deduplicated on ingest
region / device	VARCHAR	Normalised to title-case on load
ping / score / kills / deaths / duration	INT / FLOAT	Validated: duration > 0, all fields >= 0
score_per_second	FLOAT	DERIVED — score ÷ duration
kd_ratio	FLOAT	DERIVED — kills ÷ max(deaths, 1)

players (aggregated, updated per batch)

Field	Description
total_score / kills / deaths / matches_played	Running aggregates across all clean matches
skill_score	Weighted metric: avg_score×0.5 + avg_kills×3.0 – avg_deaths×1.0
cheat_status / cheat_severity / flag_reasons	"clean" or "under_review" with severity level and triggered rules

4. Leaderboard Logic

Player stats are aggregated per match history, then sorted using a cascading tie-breaker:

Priority	Column	Direction	Rationale
1st	total_score	DESC	Higher cumulative score = better performance
2nd	total_deaths	ASC	Fewer deaths wins on equal score
3rd	total_kills	DESC	More kills wins if deaths also equal
4th	matches_played	DESC	Consistency rewarded if all else equal

Flagged players appear at the bottom with rank = None and status = "under review". Their scores do NOT shift any clean player's rank number. A separate regional leaderboard applies the same logic within each primary region group.

5. Suspicious Player Detection Logic

Two-layer approach no ML training data needed:

Layer 1 Per-match rule checks:

Rule	Condition	Severity	Example
R1 Score spike	score_per_second > 50	High	P003: 99000/60s = 1650/s
R2 K/D zero-death	kills >= 20 AND deaths == 0	High	P003: 250 kills, 0 deaths
R3 Short+high	duration < 120s AND score > 5000	Medium	Score 99000 in 60s
R4 Extreme kills	kills > 50 in one match	Medium	250 kills in one match
R5 Ping anomaly	ping < 5ms OR ping > 500ms	Low	Network/data quality flag

Layer 2 Historical Z-score (>=3 matches needed):

$z = (\text{match_value} - \text{player_mean}) \div \text{player_std}$. $|z| > 3.0$ for score OR kills → "Low" severity flag. Catches players who are normally average but had one extraordinary match. Escalation: if >= 5 matches AND > 50% trigger gameplay rules → severity overridden to High.

6. Matchmaking Logic

3-pass bucketing algorithm:

Pass	Criterion	Rule
Pass 1	Region	Group by primary region — minimises baseline ping variance
Pass 2	Ping band	Low ≤80ms Med 81-150ms High >150ms — Low/High never mixed
Pass 3	Skill tier	>=6 players: Beginner/Intermediate/Advanced (33rd percentile splits); <6: Mixed pool

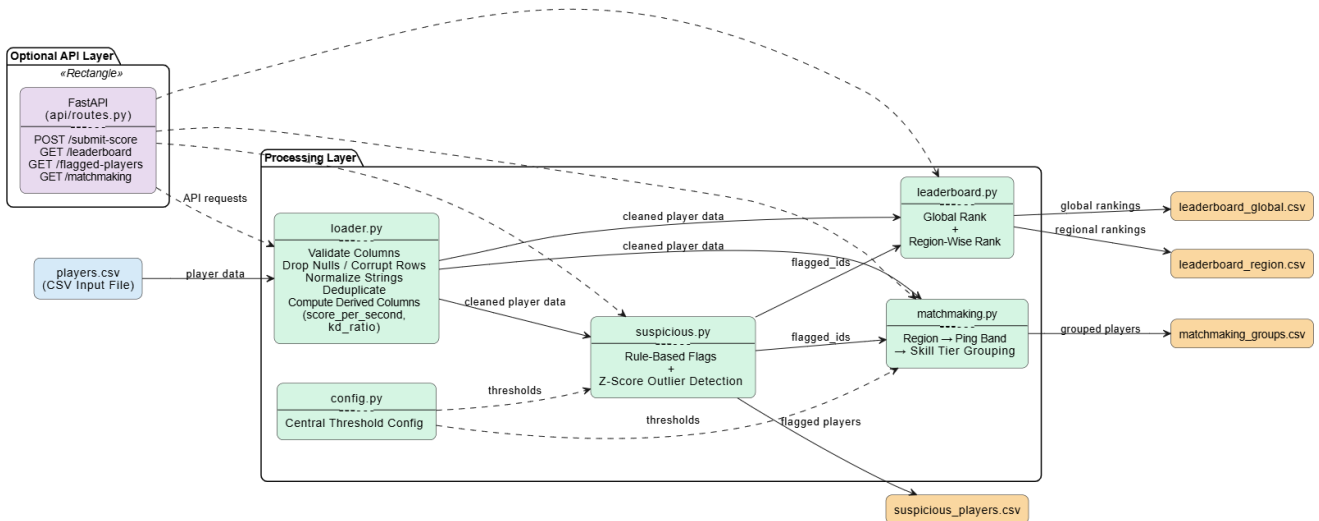
Cheater handling High severity: excluded entirely | Medium: isolated "Under_Review" lobby | Low: matched normally but tagged. Lobbies with < 4 players are marked "Waiting".

7. Architecture Explanation

Modular pipeline each concern is an isolated, independently testable Python module:

Layer	Component	Responsibility
Input	data/players.csv	Raw match telemetry — single CSV source
Loader	src/loader.py	Validate, clean, deduplicate, compute derived columns
Detection	src/suspicious.py	Rule checks (R1–R5) + Z-score history per player
Leaderboard	src/leaderboard.py	Aggregate, sort with tie-breakers, split global/regional
Matchmaking	src/matchmaking.py	Skill score, 3-pass bucketing, isolate cheaters
CLI	main.py	Orchestrate full pipeline → 4 CSV reports in output/
API	api/ (FastAPI + uvicorn)	POST /submit-score GET /leaderboard /flagged-players /matchmaking /download
Dashboard	static/ (HTML+CSS+JS)	Premium glassmorphic SPA — real-time metrics, upload, tables, lobbies

System Architecture



8. Scaling Plan

Scale	Component	Approach
2K players	Current prototype	In-memory Pandas + single FastAPI pod
20K players	Database + Cache	PostgreSQL (partitioned by region/date) + Redis Sorted Set for leaderboard reads < 5ms
200K players	Async ingestion	Kafka / AWS SQS message queue — API publishes events; async workers write to DB, decoupling write pressure
200K players	Real-time anti-cheat	Apache Flink stream job on Kafka topic — fires rule alerts in real time
200K players	Batch Z-score	Apache Spark scheduled job every 10 min over PostgreSQL match history
Any scale	API pods	Stateless FastAPI containers; Kubernetes HPA auto-scales on request rate
Any scale	Monitoring	Prometheus scrapes latency/queue lag/errors; Grafana dashboards with alert thresholds

9. Limitations

Limitation	Detail
Static thresholds	Rule limits are manually tuned; prod needs auto-calibration per game version/patch
No authentication	API endpoints have no token validation or rate limiting — required in production
Static matchmaking	One-shot bucketing; no live queue with wait-time relaxation
No persistence on restart	In-memory DataFrame lost on server restart; prod uses DB with transaction log
Z-score needs history	New players (< 3 matches) not covered by statistical layer — rule checks only
Region normalisation	"SEA" becomes "Sea" after title-case; prod needs canonical region registry
No season reset	Leaderboard is cumulative; season_id and date-window filtering needed for resets